Reviewer Pack — Minimal Order of Noncrossing Partitions and Circuit Depth In…

Entry 16db315b1aa1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 03:17:15 UTC

Minimal Order of Noncrossing Partitions and Circuit Depth Inequality

Entry ID: `16db315b1aa1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 03:17:15 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Enumeration (Noncrossing Partitions) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Depth)

Statement:

> For every CNF formula φ with n variables, the minimal order of noncrossing partitions for its associated graphic representation G(φ) is linearly correlated with its circuit depth d(φ), such that O(mtr(G(φ))) ≤ d(φ) ≤ Θ(2^(mtr(G(φ)) + 1/n^2)), where mtr(G(φ)) represents the minimal order of noncrossing partitions and d(φ) is the circuit depth.

Rationale (proposer's reasoning):

> Noncrossing partitions provide a combinatorial structure that can be used to encode Boolean functions in a way that may reveal deep connections between the structure of these partitions and the complexity of circuits computing the associated functions. This conjecture suggests that the order of noncrossing partitions could serve as an invariant for circuit depth, which has not been explored in this context.

Taxonomy category: `ENUMERATION_COMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1dc7f269309e3dba`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a CNF formula φ with n variables, if mtr(G(φ)) ≤ d(φ) ≤ Θ(2^(mtr(G(φ)) + 1/n^2)), where mtr(G(φ)) is the minimal order of noncrossing partitions and d(φ) is the circuit depth.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'noncrossing partitions' AND 'circuit depth' AND CNF` - `'minimal order' AND 'noncrossing partitions' AND 'Boolean circuit complexity'` - `correlation 'mtr(G(φ))' AND 'circuit depth d(φ)'`

Top relevant hits considered: - [http://arxiv.org/abs/1604.06009v2] Oriented Flip Graphs and Noncrossing Tree Partitions - [http://arxiv.org/abs/2212.13799v6] Noncrossing partitions of a marked surface - [http://arxiv.org/abs/1904.05573v3] k-Indivisible Noncrossing Partitions - [http://arxiv.org/abs/1801.06078v2] Noncrossing partitions, Bruhat order and the cluster complex - [http://arxiv.org/abs/0905.0205v1] Euler characteristic of the truncated order complex of generalized noncrossing partitions - [http://arxiv.org/abs/1808.08676v3] Constraining the p-mode–g-mode tidal instability with GW170817 - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.7s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.randint(-n, -1) for _ in range(random.randint(1, n))]
            if random.choice([True, False]):
                clause = [-x for x in clause]
            clauses.append(clause)
        return clauses

    def circuit_depth(cnf):
        stack = []
        for literal in cnf:
            if literal > 0:
                stack.append(literal)
            else:
                if not stack or -literal != stack.pop():
                    return float('inf')
        return len(stack)
```

```
    def mtr(graph):
        n = len(graph)
        dp = [[float('inf')] * (1 << n) for _ in range(n)]
        for i in range(n):
            dp[i][1 << i] = 1
        for mask in range(1, 1 << n):
            for i in range(n):
                if mask & (1 << i):
                    for j in range(i):
                        if graph[i][j] and mask & (1 << j):
                            dp[i][mask] = min(dp[i][mask], dp[j][mask ^ (1 << i)] + 1)
        return min(max(row) for row in dp)

    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    graph = [[False] * n for _ in range(n)]
    for clause in cnf:
        for literal in clause:
            if literal > 0:
                i = abs(literal) - 1
                for other_literal in clause:
                    j = abs(other_literal) - 1
                    graph[i][j] = True

    mtr_value = mtr(graph)
    depth = circuit_depth(cnf)

    if mtr_value <= depth <= 2**(mtr_value + 1/n**2):
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "Circuit depth does not satisfy the inequality"

    return {
        "metric_name": "circuit_depth",
        "metric_value": depth,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
```

```
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17199
2	preregistration	ollama_remote	glm4:latest	0	0	10398
3	novelty	ollama_remote	glm4:latest	0	0	8536
4	novelty	ollama_remote	glm4:latest	0	0	15460
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33529
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10509
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9811
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9871
9	judge	ollama_remote	glm4:latest	0	0	65878

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 181189 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/16db315b1aa1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/16db315b1aa1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/16db315b1aa1.tar.gz` (if generated)